
Learning Lattice Variables for Matrix-Free Wilson-Dirac Preconditioning in Lattice QCD

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Solving large, sparse, and ill-conditioned linear systems dominates the computa-
2 tional cost of lattice quantum chromodynamics (LQCD). Preconditioners effectively
3 accelerate the convergence of iterative solvers but generally need careful design
4 and tuning. This work develops an encoder–representation–decoder framework to
5 produce a matrix-free neural preconditioner for the Wilson-Dirac operator. The
6 encoder maps the gauge configuration U to a representation V , which the decoder
7 turns into a Hermitian positive-definite (HPD) preconditioner M^{-1} for the Krylov
8 solver. Because the representation consists of matrix-valued lattice fields, the
9 decoder is assembled from lattice operators and never forms the actual precondition-
10 ing matrix—a computationally prohibitive operation for typical systems. No
11 learned quantity in either the encoder or the decoder is tied to the lattice volume, so
12 a trained encoder–decoder pair applies unchanged at any lattice size. Across four
13 encoders and four decoders, we train the framework using gauge configurations
14 of lattice size $L^4 = 8^4$ against the unrolled preconditioned Conjugate Gradient
15 (PCG) objective and show that the generated preconditioners effectively reduce
16 the number of iterations required for convergence, achieving a $7.8\times$ reduction
17 compared to the unpreconditioned case. In addition, we directly apply the trained
18 encoder–decoder pairs to a larger system of $L^4 = 16^4$, where the convergence
19 behavior and relative ordering are preserved ($\sim 8\times$ iteration reduction) without
20 retraining.

21 1 Introduction

22 LQCD requires the repeated solution of large, sparse linear systems

$$A\psi = b, \quad A = (D^\dagger D)[U], \quad b = D^\dagger[U]\eta, \quad (1)$$

23 where $D[U]$ is the discretized Wilson-Dirac operator on a background gauge configuration $U =$
24 $\{U_\mu(x)\}_{x \in \Lambda, \mu=1,\dots,4}$ with $U_\mu(x) \in \text{SU}(3) \subset \mathbb{C}^{3 \times 3}$, ψ is the quark field, η is a fermion source, Λ
25 denotes the finite four-dimensional hypercubic lattice of sites, and $\mu \in \{1, \dots, 4\}$ labels a lattice
26 direction ($\hat{\mu}$ denotes the unit lattice vector in direction μ) [Wilson, 1974]. These systems are extremely
27 high dimensional ($\dim(\Lambda) \gtrsim 10^8$) and ill-conditioned in production-scale calculations, hence solving
28 them dominates the computational cost of computing observables [Joó et al., 2019].

29 Preconditioners, often denoted as M^{-1} , accelerate the convergence of solving the Dirac equation
30 by reducing the condition number of the linear system. However, classical preconditioners require
31 careful design and tuning, often incurring a substantial setup cost for each new system [Saad, 2003].
32 Meanwhile, learned preconditioners amortize this into a one-time training cost, producing M^{-1} for
33 a new system in a single forward pass [Häusner et al., 2024, Pfahler et al., 2026, Sun et al., 2026,
34 Cali et al., 2023, Lehner and Wettig, 2023]. Due to the high dimensionality, learned preconditioners

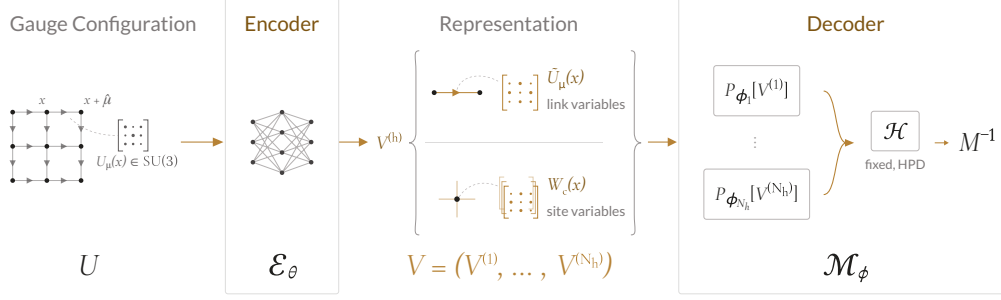


Figure 1: The encoder–representation–decoder pipeline. An encoder maps the data defining the system to a representation of site and/or link variables, which the decoder uses to build a preconditioner.

for the Dirac equation in LQCD must be matrix-free—circumventing storing the entire matrix at once—and ideally applicable across variations of the lattice dimensionality. When paired with PCG solvers, the preconditioners additionally need to be fixed linear HPD operators.

Inspired by the matrix-free neural preconditioner for the U(1) Schwinger model [Sun et al., 2026], we propose a general encoder–representation–decoder framework that constructs preconditioners for the Dirac operator associated with each SU(3) gauge configuration, shown in Figure 1. Having the unrolled PCG residuals [Nytke et al., 2025] as the training objective, this framework is effective across four encoder and four decoder architectures, significantly reducing the number of iterations needed for convergence compared to the unpreconditioned case. Trained on gauge configuration data with lattice size $L^4 = 8^4$, the framework provides up to a $7.8\times$ reduction in the number of iterations, and is immediately applicable to a larger lattice size ($L^4 = 16^4$) with up to an $8.1\times$ reduction, preserving the convergence acceleration without further training or tuning.

2 Related work

Preconditioning. Preconditioning replaces a linear system with an equivalent one that an iterative method solves faster. When using the CG solver, the iteration count grows as $O(\sqrt{\kappa(A)})$ where $\kappa(\cdot)$ is the condition number; hence, an effective preconditioner M^{-1} will be such that $M^{-1}A$ is better conditioned than A alone. Classical preconditioners such as incomplete Cholesky and even-odd (Schur complement) decomposition are built directly from the entries of A and must be rebuilt for every new system (here every gauge configuration) [Saad, 2003], a significant setup cost that affects the speed of these solves. Adaptive multigrid (AMG) [Trottenberg et al., 2000, Brandt and Livne, 2011], the current state-of-the-art for the Wilson-Dirac equation [Frommer et al., 2014, Clark et al., 2010, 2016, Brezina et al., 2006], has a setup cost (grid hierarchy, null-vector computations, and associated hyperparameters) that should be re-tuned for each new lattice size.

Learned preconditioners. Learned preconditioners replace this per-system construction with a one-time training cost, producing M^{-1} for a new system in a single forward pass [Häusner et al., 2024]. However, existing learned preconditioners fall short in different ways. Some form A or M^{-1} explicitly [Cali et al., 2023, Sappl et al., 2019], and others must be retrained for each lattice size [Häusner et al., 2024], both impractical at production scale. Recent work [Sun et al., 2026] learns a map $U_\mu(x)$ to $\tilde{U}_\mu(x)$, a new link variable, and defines $M^{-1} = (D^\dagger D)[\tilde{U}]$, which is matrix-free and transfers across volumes. However, they work only with U(1) gauge links and their preconditioner (decoder) is restricted to the same Dirac normal operator $D^\dagger D$ the system is defined by.

3 Methods

The proposed framework consists of three stages (Figure 1).

Encoder. The goal of an encoder $\mathcal{E}_\theta$ is to produce a preconditioner-generating representation of the physical gauge configuration U that determines the target Dirac equation to be solved. The encoder is parameterized by θ and maps U to the preconditioner-generating representation

$$\mathcal{E}_\theta(U) = \{V^{(1)}, \dots, V^{(N_h)}\}, \quad (2)$$

where each of the N_h heads h carries a set of matrix-valued lattice variables $V^{(h)} = \{\tilde{U}^{(h)}, W^{(h)}\}$.

Representation. $\tilde{U}^{(h)}$ is a set of link matrices $\{\tilde{U}_\mu^{(h)}(x) \in \mathbb{C}^{3 \times 3}\}_{x \in \Lambda, \mu=1, \dots, 4}$, and $W^{(h)}$ is a set of site matrices $\{W_c^{(h)}(x) \in \mathbb{C}^{3 \times 3}\}_{x \in \Lambda, c=1, \dots, N_c}$. These are exactly the objects the standard lattice operations (e.g., parallel transport) consume, so the decoder can be assembled from them without ever forming a matrix. We require neither linearity nor $\text{SU}(3)$ representations since linearity is required only of the preconditioner (which the decoder fully controls) and the lattice operators of the decoder do not require unitary inputs.

Decoder. The role of a decoder $\mathcal{M}_\phi$ in the proposed framework is to construct linear operators P_{ϕ_h} from the representations $V^{(h)}$, which are the building blocks of M^{-1} :

$$M^{-1} = \mathcal{H}\left(P_{\phi_1}[V^{(1)}], \dots, P_{\phi_{N_h}}[V^{(N_h)}]\right), \quad (3)$$

where $P_{\phi_1}[V^{(1)}], \dots, P_{\phi_{N_h}}[V^{(N_h)}] = \mathcal{M}_\phi(V^{(1)}, \dots, V^{(N_h)})$ and $\mathcal{H}$ is a fixed (not learned) function chosen so that M^{-1} is HPD. The (possibly empty) learnable parameters are $\phi = (\phi_1, \dots, \phi_{N_h})$, where ϕ_h is used only by P_{ϕ_h} and hence only on head h .

The encoder and decoder are jointly trained unsupervised using the unrolled PCG objective [Nytko et al., 2025],

$$\psi_K(\theta, \phi) = \text{PCG}\left(A, b; M_{\theta, \phi}^{-1}[U], K\right), \quad r_K(\theta, \phi) = b - A\psi_K(\theta, \phi), \quad \mathcal{L}(\theta, \phi) = \frac{\|r_K(\theta, \phi)\|_2}{\|b\|_2}, \quad (4)$$

where PCG is run for K steps (a hyperparameter), $\mathcal{L}$ is the loss, r_K is the PCG residual after K steps, and A and b define the linear system in Eq. (1).

For simplicity, we drop the head index when $N_h = 1$, writing $\mathcal{E}_\theta(U) = V$ and $M^{-1} = \mathcal{H}(P_\phi[V])$.

4 Numerical Experiments

We use $3 + 1$ dimensional hypercubic lattices of size L^4 with lattice spacing set to unity for simplicity. Gauge configurations are generated from the Wilson plaquette action [Wilson, 1974, Gattringer and Lang, 2009] using a heatbath algorithm implemented in `GaugeFields.jl` [Nagai and Tomiya, 2024], with the coupling $\beta = 6.0$. For the Dirac operator, we set the hopping parameter $\kappa = 0.155$, near the critical value $\kappa_c = 0.158$ for this lattice size and coupling. We train and evaluate the proposed framework with different encoder-decoder pairs on $L = 8$ lattices, and test zero-shot transfer by evaluating models trained exclusively on $L = 8$ directly on $L = 16$.

We compare four choices for the encoder $\mathcal{E}_\theta$. The **identity map**, $V = \mathcal{E}_\theta(U) = \{U\}$ (a singleton), passes the physical links directly to the decoder and serves to isolate its effects. The **CNN** of Sun et al. [2026], extended here to $3 + 1$ dimensions and $\text{SU}(3)$, is a multi-head ($N_h = 3$) fully convolutional network that produces $V^{(h)} = \{\tilde{U}^{(h)}\}$. The remaining two, **L-CNN** and **site-L-CNN**, are variants of the lattice gauge-equivariant CNN of Favoni et al. [2022], which constructs gauge-equivariant features from untraced Wilson loops and parallel-transport convolutions; the L-CNN is a single-head model that produces link variables $V = \{\tilde{U}\}$ with each $\tilde{U}_\mu(x) \in \text{SU}(3)$, whereas the site-L-CNN (also $N_h = 1$) outputs each of its layer’s hidden activations, giving $V = \{W\}$ (see Appendix A).

We evaluate four decoder architectures $\mathcal{M}_\phi$. The **WNO** decoder is parameter-free, using $P[V^{(h)}] = (D^\dagger D)[V^{(h)}]$, the Wilson-Dirac normal operator built from head h ; setting $N_h = 1$ recovers the decoder used in Sun et al. [2026], but we find best results with $N_h = 3$. The **HMP** decoder uses a polynomial in the hopping operator $H[V^{(h)}]$ [Gattringer and Lang, 2009, Sec. 5.3.1], $f(H[V^{(h)}])$, fed into $P_{\phi_h}[V^{(h)}] = f^\dagger f$. Both decoders use a palindromic composition of $(P[V^{(h)}], 1 \leq h \leq N_h)$

Table 1: PCG iterations (speedup) at $\beta = 6.0$, $\kappa = 0.155$, averaged over 400 solves from 20 gauge configurations and 20 right-hand sides each. All models except AMG_{16} are trained or tuned on $L = 8$; AMG_{16} is re-tuned on $L = 16$. Speedup is relative to unpreconditioned CG. Lower iterations and higher speedup are better. The lower block lists encoder $\rightarrow$ decoder pairs built with our framework. Of these, $\text{CNN} \rightarrow \text{WNO}$ is the preconditioner of Sun et al. [2026] extended to $\text{SU}(3)$ and $3+1$ dimensions, and $\text{Identity} \rightarrow \text{PTLN}$ corresponds most closely to the model of Pfahler et al. [2026]; the other three pairs are new combinations enabled by the framework.

Encoder $\rightarrow$ Decoder	$L = 8$	$L = 16$
None (Unpreconditioned)	436 (1.0 $\times$)	1331 (1.0 $\times$)
AMG_8	117 (3.7 $\times$)	409 (3.3 $\times$)
AMG_{16}	–	264 (5.0 $\times$)
$\text{L-CNN} \rightarrow \text{PTLN}$	56 (7.8 $\times$)	165 (8.1 $\times$)
$\text{Identity} \rightarrow \text{PTLN}$	56 (7.8 $\times$)	168 (7.9 $\times$)
$\text{Site-L-CNN} \rightarrow \text{CM-PTLN}$	57 (7.6 $\times$)	170 (7.8 $\times$)
$\text{CNN} \rightarrow \text{HMP}$	277 (1.6 $\times$)	841 (1.6 $\times$)
$\text{CNN} \rightarrow \text{WNO}$	278 (1.6 $\times$)	841 (1.6 $\times$)

for $\mathcal{H}$. The remaining two decoders both use $N_h = 1$ and admit exact adjoints which we use in $\mathcal{H}(P_\phi[V]) = P_\phi[V]^\dagger P_\phi[V]$. The **PTLN** [Pfahler et al., 2026] uses V -dependent parallel-transport layers with prescribed paths and learnable linear layers; we fix the path set at $L = 8$ and reuse it unchanged at $L = 16$ so nothing changes between lattice sizes. Finally, the **CM-PTLN** adds color mixing to the PTLN’s linear layers using the site-L-CNN’s final-layer output $\{W_c^{(N_\ell)}(x)\}$. See Appendix B for more decoder construction details.

We train five combinations of the listed encoders and decoders using the objective in Eq. (4), setting $K = 20$. Implementation details, model hyperparameters, and training set up are in Appendix C. We evaluate preconditioners by the number of PCG iterations required for convergence, averaged over a set of gauge configurations from held-out $L^4 = 8^4$ data and new $L^4 = 16^4$ data.

Table 1 summarizes the results. All learned preconditioners improve over unpreconditioned CG and also outperform AMG in this particular small volume setting. More importantly, their relative gains are preserved under zero-shot transfer from $L = 8$ to $L = 16$, with the best learned models achieving approximately $8\times$ speedup at both lattice sizes. In contrast, AMG degrades from $3.7\times$ to $3.3\times$ when transferred without modification and requires re-tuning on $L = 16$ to recover to $5.0\times$.

The most effective preconditioner on both $L = 8$ and $L = 16$ is $\text{L-CNN} \rightarrow \text{PTLN}$, although its improvement over $\text{Identity} \rightarrow \text{PTLN}$ is small. Both require 56 iterations at $L = 8$, while at $L = 16$ they require 165 and 168 iterations, respectively. Inspection of the learned representations suggests why this margin is so narrow: During training, the L-CNN encoder converges close to the identity map, i.e. the $\text{L-CNN} \rightarrow \text{PTLN}$ effectively becomes $\text{Identity} \rightarrow \text{PTLN}$ (see Appendix D).

5 Conclusion

We present an encoder–representation–decoder framework for training neural preconditioners for the Wilson–Dirac operator. Trained with an unsupervised unrolled PCG residual objective, the encoder takes the lattice gauge configuration as input and produces a representation that is used by the decoder to construct matrix-free preconditioners. Across five encoder–decoder pairs with different architectures, we show that the proposed framework reduces the number of PCG iterations required for convergence and transfers zero-shot to a larger lattice size without additional training. Future work will explore additional encoder–decoder architectures, investigate the learned preconditioners through spectral analysis, and evaluate the framework at production-scale lattice sizes.

References

- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

143 Achi Brandt and Oren E Livne. *Multigrid techniques*. Society for Industrial and Applied Mathematics,
144 January 2011.

145 M Brezina, R Falgout, S MacLachlan, T Manteuffel, S McCormick, and J Ruge. Adaptive algebraic
146 multigrid. *SIAM J. Sci. Comput.*, 27(4):1261–1286, January 2006.

147 Salvatore Cali, Daniel C. Hackett, Yin Lin, Phiala E. Shanahan, and Brian Xiao. Neural-network
148 preconditioners for solving the Dirac equation in lattice gauge theory. *Phys. Rev. D*, 107:034508,
149 2023. doi: 10.1103/PhysRevD.107.034508.

150 M. A. Clark, R. Babich, K. Barros, R. C. Brower, and C. Rebbi. Solving lattice QCD systems of
151 equations using mixed precision solvers on GPUs. *Computer Physics Communications*, 181(9):
152 1517–1528, 2010. doi: 10.1016/j.cpc.2010.05.002.

153 M. A. Clark, Bálint Joó, Alexei Strelchenko, Michael Cheng, Arjun Gambhir, and Richard Brower.
154 Accelerating lattice QCD multigrid on GPUs using fine-grained parallelization, 2016.

155 DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter
156 Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Clau-
157 dio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel,
158 Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch,
159 Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John
160 Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider,
161 Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec,
162 Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL
163 <http://github.com/google-deeppmind>.

164 Matteo Favoni, Andreas Ipp, David I. Müller, and Daniel Schuh. Lattice gauge equivariant convolu-
165 tional neural networks. *Phys. Rev. Lett.*, 128:032003, 2022. doi: 10.1103/PhysRevLett.128.032003.

166 A Frommer, K Kahl, S Krieg, B Leder, and M Rottmann. Adaptive aggregation-based domain
167 decomposition multigrid for the lattice Wilson–Dirac operator. *SIAM J. Sci. Comput.*, 36(4):
168 A1581–A1608, January 2014.

169 Jean H. Gallier. Basics of classical lie groups: The exponential map, lie groups, and lie algebras. In
170 *Geometric Methods and Applications*, volume 38 of *Texts in Applied Mathematics*, pages 367–414.
171 Springer, 2001. doi: 10.1007/978-1-4613-0137-0_14.

172 Christof Gattringer and Christian B Lang. *Quantum chromodynamics on the lattice*. Lecture Notes in
173 Physics. Springer, Berlin, Germany, 2010 edition, October 2009.

174 Paul Häusser, Ozan Öktem, and Jens Sjölund. Neural incomplete factorization: learning precondi-
175 tioners for the conjugate gradient method, 2024.

176 Bálint Joó, Chulwoo Jung, Norman H. Christ, William Detmold, Robert G. Edwards, Martin Savage,
177 and Phiala Shanahan. Status and future perspectives for lattice gauge theory calculations to
178 the exascale and beyond. *The European Physical Journal A*, 55(11), 2019. doi: 10.1140/epja/
179 i2019-12919-7.

180 Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and
181 filtered transformations. *Differentiable Programming workshop at Neural Information Processing
182 Systems 2021*, 2021.

183 Christoph Lehner and Tilo Wettig. Gauge-equivariant neural networks as preconditioners in lattice
184 QCD. *Phys. Rev. D*, 108:034503, 2023. doi: 10.1103/PhysRevD.108.034503.

185 Yuki Nagai and Akio Tomiya. Juliaqcd: Portable lattice qcd package in julia language, 2024. URL
186 <https://arxiv.org/abs/2409.03030>.

187 Nicolas Nytko, Ali Taghibakhshi, Tareq Uz Zaman, Scott MacLachlan, Luke N. Olson, and Matthew
188 West. Optimized sparse matrix operations for reverse mode automatic differentiation. *SIAM J. Sci.
189 Comput.*, 47(5):C1115–C1143, 2025.

- 190 Simon Pfahler, Daniel Knüttel, Christoph Lehner, and Tilo Wettig. A novel gauge-equivariant neural
 191 network architecture for preconditioners in lattice QCD. In *Proceedings of The 42nd International*
 192 *Symposium on Lattice Field Theory — PoS(LATTICE2025)*, page 027. SISSA Medialab, 2026. doi:
 193 10.22323/1.518.0027.
- 194 Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, PA, 2 edition, 2003.
- 195 Johannes Sappl, Laurent Seiler, Matthias Harders, and Wolfgang Rauch. Deep learning of precondi-
 196 tioners for conjugate gradient solvers in urban water related problems, 2019.
- 197 Yixuan Sun, Srinivas Eswar, Yin Lin, William Detmold, Phiala Shanahan, Yang Liu, Xiaoye Li, and
 198 Prasanna Balaprakash. Matrix-free neural preconditioner for the dirac operator in lattice gauge
 199 theory. *ACM Trans. AI Sci.*, August 2026. doi: 10.1145/3841473. URL [https://doi.org/10.](https://doi.org/10.1145/3841473)
 200 1145/3841473. Just Accepted.
- 201 Akio Tomiya, Hiroshi Ohno, and Yuki Nagai. CASK: A gauge covariant transformer for lattice
 202 gauge theory. In *Proceedings of The 41st International Symposium on Lattice Field Theory —*
 203 *PoS(LATTICE2024)*, page 030, Trieste, Italy, January 2025. Sissa Medialab.
- 204 Ulrich Trottenberg, Cornelius W Oosterlee, and Anton Schuller. *Multigrid*. Academic Press, San
 205 Diego, CA, November 2000.
- 206 Kenneth G. Wilson. Confinement of quarks. *Phys. Rev. D*, 10:2445–2459, 1974. doi: 10.1103/
 207 PhysRevD.10.2445.

208 A L-CNN Architecture and Variants

209 A.1 Changes to the Original L-CNN

210 A lattice gauge-equivariant CNN (L-CNN) [Favoni et al., 2022] builds gauge equivariance into the
 211 architecture rather than learning it. It operates on a pair (U, W) , where $U = \{U_\mu(x) \in \text{SU}(3)\}$ are
 212 link variables and $W = \{W_c(x) \in \mathbb{C}^{3 \times 3}\}$ are $N_c = N_{\text{ch}}$ channels of matrix-valued site variables.
 213 The channels are initialized with untraced Wilson loops built from the links and refined by a stack
 214 of gauge-equivariant blocks that parallel transport them between neighboring sites and take learned
 215 products of them. These can form Wilson loops of arbitrary shape and size and make L-CNNs
 216 universal approximators of gauge-equivariant functions on the lattice [Favoni et al., 2022].

217 Our use of the L-CNN differs from the original in two respects:

- 218 1. **The output is a gauge field, not a scalar.** Favoni et al. [2022] terminate their networks with
 219 a trace layer, producing scalars for each site that a conventional CNN then regresses onto a
 220 physical observable. We do not use a trace layer, but instead terminate with their (proposed,
 221 but unused) L-Exp layer,

$$\tilde{U}_\mu(x) = \exp\left(i \sum_c \beta_{\mu,c} [W_c(x)]_{\text{ah}}\right) U_\mu(x), \quad (\text{A.1})$$

222 where $[\cdot]_{\text{ah}}$ is the traceless anti-Hermitian part and the $\beta_{\mu,c} \in \mathbb{R}$ are learned. The exponent
 223 lies in $\mathfrak{su}(3)$, so $\tilde{U}_\mu(x) \in \text{SU}(3)$ is guaranteed [Gallier, 2001, Theorem 14.4.2]. The whole
 224 model is a gauge-equivariant map from $\text{SU}(3)$ links to $\text{SU}(3)$ links, with a single head
 225 ($N_h = 1$), so its representation is $V = \{\tilde{U}\}$ and it is compatible with all of our decoders
 226 (Appendix B). We initialize $\beta_{\mu,c} \sim \mathcal{N}(0, 0.1^2)$, a small perturbation of the identity map
 227 ($\beta_{\mu,c} = 0 \ \forall \mu, c$ reproduces $\tilde{U} = U$).

- 228 2. **A gauge-equivariant normalization layer.** We insert a per-channel RMS normalization
 229 after each block,

$$W_c = \frac{W_c}{\sqrt{\text{mean}_x \|W_c(x)\|_F^2 + \varepsilon}} \cdot \gamma_c, \quad (\text{A.2})$$

230 where $\gamma_c \in \mathbb{R}$ is a learnable scalar and $\|\cdot\|_F$ is the Frobenius norm. The Frobenius norm
 231 is gauge invariant [Tomiya et al., 2025], so rescaling by it preserves equivariance. This
 232 stabilizes training and prevents the exploding activations caused by the repeated products
 233 that build up the channels.



Figure A.1: The L-CNN architecture used with added L-Exp and RMSnorm layers.

A.2 Site-L-CNN

The **site-L-CNN** is identical to the L-CNN in Appendix A.1, except that it omits the L-Exp layer and outputs the site variables from every layer. It has a single head ($N_h = 1$), and its output representation is

$$V = \{W\} = \{W_c(x)\}_{x \in \Lambda, 1 \leq c \leq N_c}, \quad W_c(x) \in \mathbb{C}^{3 \times 3}, \quad (\text{A.3})$$

where $N_c = N_{\text{ch}} \cdot N_\ell$ is the number of channels per layer times the number of layers. These are the site variables consumed by the CM-PTLN decoder (Appendix B).

B Decoder construction

Here we go into more detail on some of the decoders we use.

Wilson Normal Operator (WNO). This parameter-free decoder uses $P[V^{(h)}] = (D^\dagger D)[V^{(h)}]$ (the Wilson normal operator) and

$$M^{-1} = \mathcal{H}(P[V^{(1)}], \dots, P[V^{(N_h)}]) = P[V^{(1)}] \cdot P[V^{(2)}] \dots P[V^{(N_h)}] \dots P[V^{(2)}] \cdot P[V^{(1)}], \quad (\text{B.1})$$

which is HPD by construction. When using link variable representations $\tilde{U}^{(h)}$ and $N_h = 1$, this recovers the decoder of Sun et al. [2026].

Hopping Matrix Polynomial (HMP). We can write the Wilson-Dirac operator as $D[V] = 1 - \kappa H[V]$, where $H[V]$ is the *hopping matrix* [Gattringer and Lang, 2009, Section 5.3.1] and κ is the hopping parameter. The Neumann series $D[V]^{-1} = \sum_{j \geq 0} \kappa^j H[V]^j$ then converges for $\kappa < 1/8$ [Gattringer and Lang, 2009, Section 5.3.1], which our value of $\kappa = 0.155$ falls outside of. This suggests instead trying a *learnable, truncated* polynomial in the hopping matrix,

$$f(V^{(h)}) = \sum_{j=0}^n a_{h,j} \kappa^j H[V^{(h)}]^j, \quad P_{\phi_h}[V^{(h)}] = f(V^{(h)})^\dagger f(V^{(h)}), \quad (\text{B.2})$$

$$\phi_h = \{a_{h,j}\}_{j=0}^n \subset \mathbb{R} \text{ learnable},$$

with the $\mathcal{H}$ operator from Eq. (B.1), making the preconditioner HPD by construction. We find best results with $n = 1$.

Parallel-Transport Linear Network (PTLN). The PTLN [Pfahler et al., 2026] is built from two layer types: a **parallel transport layer (PT)** applies a chain of single hops along a fixed (hardcoded) path to each of its input fields, acting on the color indices, and a traditional **linear layer (L)** then transforms the spinor indices; the linear layer weights are the only trainable parameters. Straying from Pfahler et al. [2026], we construct the exact adjoint $P_\phi^\dagger$ by reversing the order of the layers, conjugate-transposing each linear layer weight matrix, and reversing and flipping each parallel transport path; we then use $\mathcal{H}(P_\phi[V]) = P_\phi[V]^\dagger P_\phi[V]$, which is HPD by construction. Importantly, we have the PT layers transport along the representation $V = \mathcal{E}_\theta(U)$ rather than along the physical links U .

Color-Mixing PTLN (CM-PTLN). This model replaces each PTLN linear layer weight S with a sum of Kronecker products $\sum_c S_c \otimes W_c^{(N_\ell)}(x)$, in which S_c acts on the spinor indices as before while the site-L-CNN's final-layer output $W_c^{(N_\ell)}(x)$ acts on the color indices. This lets the linear layers mix the three colors at a site, something the PTLN's linear layers cannot do.

C Model hyperparameters and training details

C.1 Implementation

The framework is implemented in JAX with Equinox modules and Optax optimizers [Bradbury et al., 2018, Kidger and Garcia, 2021, DeepMind et al., 2020]. All lattice operators (the Wilson-Dirac operator $D[U]$, the hopping matrix $H[U]$, parallel transport, and the L-CNN layers) are written as matrix-free matrix vector products, so no preconditioner is ever assembled. Boundary conditions are periodic in the three spatial directions and antiperiodic in time. Training runs data-parallel on a single node with eight Nvidia A100 GPUs.

C.2 Data

Gauge configurations are generated with the heatbath algorithm of `Gaugefields.jl` [Nagai and Tomiya, 2024] from the Wilson plaquette action [Wilson, 1974, Gattringer and Lang, 2009] at $\beta = 6.0$, using independent Markov chains run in parallel, 100 thermalization sweeps per chain, and 10 sweeps discarded between saved configurations. The $L = 8$ ensemble is split 80/20 into training and validation sets by a fixed random permutation. The $L = 16$ ensemble is generated by the same procedure and is used only for zero-shot evaluation; no model sees $L = 16$ data during training. The hopping parameter is $\kappa = 0.155$ for both training and evaluation.

C.3 Model Hyperparameters

The hyperparameters used for the encoders and decoders are listed in Tables C.1 and C.2, respectively.

Table C.1: Hyperparameters used for encoders.

Encoder	Hyperparameters
Identity	None (no parameters)
CNN	4 complex 4D convolutions, kernel 3^4 , hidden width 16, PReLU; 36 input and output channels (4 directions $\times$ 3×3 color entries); a 3-layer shared backbone feeding 3 output heads, each producing its own $\tilde{U}^{(h)}$
L-CNN	3 (L-Conv + L-Bilin) blocks, each with 8 channels, kernel size 2 (single hop); gauge-equivariant RMS norm after each block ($\varepsilon = 10^{-6}$); channels seeded with the 6 plaquettes and 4 Polyakov loops; L-Exp read-out with $\beta_{\mu,c}$ initialized to $\mathcal{N}(0, 0.1^2)$
Site-L-CNN	Same as the L-CNN but without the L-Exp read-out

Table C.2: Hyperparameters used for decoders.

Decoder	Hyperparameters
WNO	$(D^\dagger D)[V]$; no parameters of its own
HMP	We use a polynomial of degree $n = 1$, so two learnable real coefficients a_0, a_1 per head
PTLN	3 PT layers with 25 paths each (the identity plus shifts of $\pm 1, \pm 2, \pm 4$ in each of the 4 directions) giving 2 hidden linear blocks and layer widths $1 \rightarrow 25 \rightarrow 25 \rightarrow 25 \rightarrow 1$; each linear layer acts as a complex 4×4 matrix on the spinor indices; weights initialized as $\mathcal{N}(0, (10^{-3})^2)$; a channel-0 residual skip follows every linear layer, including the output layer
CM-PTLN	$N_{\text{ch}} = 8$ in the color-mixing linear layer, matching the number of channels in the site-L-CNN; otherwise identical to the PTLN

C.4 Training

All five pairs are trained with the unrolled PCG objective of Eq. (4) with $K = 20$ steps. At every optimizer step we draw 8 fresh complex Gaussian right-hand sides per gauge configuration, run K

287 steps of PCG, and average $\|r_K\|/\|b\|$ over right-hand sides and over the batch. No right-hand sides
 288 are reused between epochs.

Table C.3: Hyperparameters used for training.

Setting	Value
Optimizer	Adam
Peak learning rate	10^{-3}
Schedule	Cosine decay over 10,000 steps
Batch size	32 gauge configurations (4 per GPU)
Unrolled PCG steps K	20
Right-hand sides per step	8
Early-stopping patience	10 epochs

289 The validation loss is the same unrolled-PCG residual evaluated on the held-out split, computed once
 290 per epoch. Training stops when it fails to improve on the best value by at least 1% for patience
 291 consecutive epochs, and the checkpoint with the lowest validation loss is the one evaluated. The
 292 epoch cap is never reached: every run early-stops within 20–60 epochs, or roughly 30 minutes to a
 293 few hours of wall time.

294 C.5 Evaluation

295 Test-time solves use PCG on $A = (D^\dagger D)[U]$ with $b = D^\dagger[U] \eta$ for complex Gaussian sources η , run
 296 in double precision to a relative residual tolerance of $\|r\|/\|b\| < 10^{-8}$ with no iteration cap; every
 297 solve reported in Table 1 converged. The encoder forward pass producing V is run once per gauge
 298 configuration and reused across all right-hand sides for that configuration.

299 D Analysis of trained L-CNN

300 We find that during training of the L-CNN $\rightarrow$ PTLN model, extra gauge-equivariant capacity (the
 301 L-CNN) added to the model (the PTLN) is discarded in favor of an effectively simpler architecture.
 302 Although the L-CNN is not initialized to the identity mapping, it converges during training to
 303 L-CNN(U) $\approx U$. We track this *deviation from identity* as

$$\text{deviation}(\tilde{U}) = \text{mean}_{x, \mu} \frac{\|\tilde{U}_\mu(x) - U_\mu(x)\|_F}{\|U_\mu(x)\|_F}. \quad (\text{D.1})$$

304 If $\tilde{U}_\mu(x) \approx U_\mu(x)$ on average, then deviation ≈ 0 . From Figure D.1, we see that the L-CNN $\rightarrow$
 305 PTLN model becomes essentially equivalent to the simpler Identity $\rightarrow$ PTLN model.

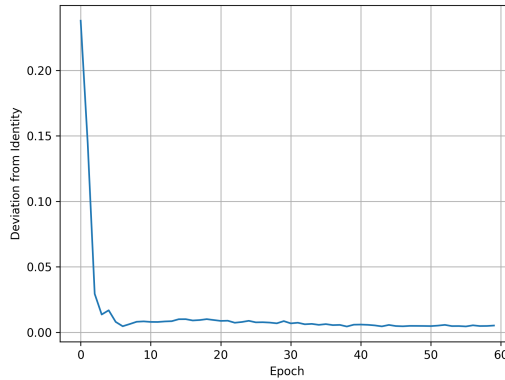


Figure D.1: Deviation from identity during training for an L-CNN encoder paired with a PTLN decoder. The L-CNN converges close to the identity mapping, showing that extra gauge-equivariant capacity is discarded in favor of a simpler architecture.